

SOFTWARE REQUIREMENTS SPECIFICATION

Job Trends Data Engineering Pipeline

Version 1.0 | April 2026 | Confidential

ELT Architecture · Azure ADLS Gen2 · DuckDB · dbt · Apache Airflow · PostgreSQL

Document Scope

This SRS defines the functional and non-functional requirements for the Job Trends Data Engineering Pipeline — an ELT system that ingests job listing data from CSV job boards and REST APIs, transforms it through Bronze, Silver, and Gold Azure Data Lake Storage layers using DuckDB and dbt, and serves aggregated job market trends via PostgreSQL and a BI dashboard. Requirements follow the "The system shall..." format and adhere to SMART criteria. Each requirement is assigned a MoSCoW priority. The document also includes use case descriptions with scenarios, and all architectural diagrams produced for the project.

Priority legend: **High** = Must Have **Medium** = Should Have **Low** = Could Have

1. Functional Requirements

The requirements below define all observable, testable behaviours the pipeline system must exhibit. Each entry uses the form "The system shall..." to express a binding obligation.

1.1 Data Ingestion

ID	Requirement (The system shall...)	Rationale	Priority
FR-01	The system shall ingest raw job listing data from CSV files exported by external job board sources on a daily scheduled basis.	CSV files are the primary offline data channel; daily ingestion maintains data freshness.	High
FR-02	The system shall ingest real-time job listing data by calling external REST API endpoints, applying exponential-backoff retry logic upon transient HTTP failures.	Retry logic ensures completeness without manual intervention.	High
FR-03	The system shall parse and validate each ingested record against a defined Pydantic schema before any downstream processing, enforcing mandatory field presence and data-type constraints.	Early validation prevents corrupted data from propagating through the pipeline.	High
FR-04	The system shall route all records that fail schema validation to a dedicated dead-letter store, preserving the original record content and the reason for rejection.	Dead-letter storage enables auditability and manual reprocessing of rejected records without data loss.	High
FR-05	The system shall land all validated records from both CSV and API sources as raw Parquet files in the Bronze layer of Azure Data Lake Storage Gen2.	Parquet columnar format enables efficient downstream reads; Bronze layer preserves the raw state for reprocessing.	High
FR-06	The system shall process CSV files in configurable chunk sizes to limit peak memory consumption during ingestion.	Chunked reading prevents out-of-memory failures on large CSV files.	Medium

1.2 Data Transformation

ID	Requirement (The system shall...)	Rationale	Priority
FR-07	The system shall execute dbt Silver transformation models using DuckDB as the SQL execution engine, reading Bronze Parquet files directly from ADLS and producing deduplicated, cleaned, source-staged Silver Parquet files.	DuckDB provides serverless, high-performance SQL execution without a persistent cluster; dbt enforces modular, testable transformations.	High
FR-08	The system shall deduplicate job records within the Silver transform layer, retaining the most recently	Duplicate records from overlapping CSV and	High

ID	Requirement (The system shall...)	Rationale	Priority
	ingested record when duplicate job identifiers are detected across sources.	<i>API sources would distort trend analytics.</i>	
FR-09	The system shall execute dbt Gold aggregation models to UNION all Silver sources and produce exactly five aggregated Gold tables covering job titles, locations, salary bands, skills demand, and posting trends.	<i>Gold tables provide the pre-aggregated datasets required by the BI dashboard, minimising query-time computation.</i>	High
FR-10	The system shall write all Silver and Gold transformed outputs as Parquet files to their respective ADLS layers before any downstream step proceeds.	<i>Materialising intermediate layers provides checkpoint recovery points and supports independent re-runs.</i>	Medium

1.3 Data Serving & Dashboard

ID	Requirement (The system shall...)	Rationale	Priority
FR-11	The system shall load all five Gold tables into a PostgreSQL serving database using a truncate-and-reload strategy on every pipeline run.	<i>Truncate-reload guarantees that the serving layer always reflects the latest full Gold snapshot without partial-update anomalies.</i>	High
FR-12	The system shall expose job trend data through a BI dashboard that allows end-users to filter job listings by title, location, date range, and salary band.	<i>Interactive filtering is the primary analytical use case for the Data Analyst actor.</i>	High
FR-13	The system shall display aggregated trend visualisations on the dashboard, including time-series charts of job posting volumes, geographic heat maps, and top-skills frequency rankings.	<i>Visual aggregations communicate trends that are not apparent from raw tabular data.</i>	High

1.4 Orchestration & Operations

ID	Requirement (The system shall...)	Rationale	Priority
FR-14	The system shall orchestrate the full pipeline as a directed acyclic graph (DAG) managed by Apache Airflow, triggered automatically once per day.	<i>A DAG enforces step ordering, dependency resolution, and provides a full audit log of each run.</i>	High
FR-15	The system shall allow a Data Engineer to manually trigger a backfill run for any specified historical date range from the Airflow UI without modifying pipeline code.	<i>Backfill capability is necessary for recovering from upstream data outages or schema changes.</i>	Medium

ID	Requirement (The system shall...)	Rationale	Priority
FR-16	The system shall expose pipeline run status, task-level logs, and dead-letter record counts to the Data Engineer through the Airflow monitoring interface.	<i>Operational visibility enables rapid diagnosis and resolution of pipeline failures.</i>	Medium

2. Non-Functional Requirements

Non-functional requirements define the quality attributes the system must satisfy. They are categorised using the ISO 25010 quality model and include measurable acceptance thresholds.

2.1 Performance & Efficiency

ID	Requirement (The system shall...)	Rationale	Priority
NFR-01	The system shall complete the full daily pipeline run from ingestion trigger to PostgreSQL load completion within a four-hour processing window.	<i>A four-hour window ensures dashboard data is refreshed and available to analysts by the start of the working day.</i>	High
NFR-02	The system shall sustain ingestion throughput of at least 500,000 job records per run without exceeding available memory, by processing CSV files in configurable chunks and streaming API responses.	<i>Chunked processing decouples memory consumption from input file size, enabling scalable ingestion.</i>	High
NFR-03	The system shall execute DuckDB SQL transformations directly against ADLS-resident Parquet files without staging data to a local disk, keeping transformation latency below 30 minutes for up to 5 million records.	<i>In-place Parquet execution eliminates redundant data movement and reduces end-to-end latency.</i>	Medium

2.2 Reliability & Fault Tolerance

ID	Requirement (The system shall...)	Rationale	Priority
NFR-04	The system shall guarantee that no validated record is silently discarded; every record that fails validation must appear in the dead-letter store with its rejection reason before the pipeline proceeds.	<i>Silent data loss is undetectable and would produce incorrect trend analytics.</i>	High
NFR-05	The system shall produce an idempotent pipeline such that re-executing any run for the same logical date produces identical output in the Gold and PostgreSQL layers.	<i>Idempotency enables safe retries and backfills without manual cleanup steps.</i>	High
NFR-06	The system shall achieve a pipeline success rate of at least 99% over any rolling 30-day period, excluding failures caused by prolonged upstream API outages exceeding four hours.	<i>High reliability is required to maintain daily dashboard freshness for business stakeholders.</i>	High

2.3 Security & Access Control

ID	Requirement (The system shall...)	Rationale	Priority
NFR-07	The system shall authenticate all access to Azure Data Lake Storage Gen2 using managed identities or service-principal credentials stored in a secrets	<i>Credential exposure in code repositories is a leading cause of data breaches.</i>	High

ID	Requirement (The system shall...)	Rationale	Priority
	manager; no credentials shall be embedded in pipeline code or configuration files.		High
NFR-08	The system shall enforce role-based access control such that Data Analysts can read Gold tables and the dashboard, but cannot access Bronze or Silver layers or modify pipeline configuration.	<i>Least-privilege access reduces the blast radius of accidental or malicious data modification.</i>	

2.4 Maintainability & Observability

ID	Requirement (The system shall...)	Rationale	Priority
NFR-09	The system shall version-control all dbt models, Airflow DAG definitions, and Pydantic schemas in a source-code repository, with changes deployable via a CI/CD pipeline without manual file copies.	<i>Version control and CI/CD are prerequisites for repeatable, auditable change management.</i>	Medium
NFR-10	The system shall emit structured logs for every pipeline task including record counts, rejected-row counts, transformation durations, and load row counts in a format queryable by the Airflow monitoring interface.	<i>Structured logs enable automated alerting and post-mortem analysis without parsing free-text output.</i>	Medium
NFR-11	The system shall be architected such that adding a new data source requires changes only to the ingestion and Silver dbt layers, with no modifications required to Gold models or the serving layer.	<i>Source isolation limits the scope of change when onboarding new job boards.</i>	Low

2.5 Scalability & Portability

ID	Requirement (The system shall...)	Rationale	Priority
NFR-12	The system shall be deployable on infrastructure that can scale DuckDB compute and ADLS storage independently, without requiring changes to pipeline code when data volumes increase by up to ten times the baseline.	<i>Compute and storage decoupling is fundamental to cloud-native scalability.</i>	Medium

3. Use Case Descriptions and Scenarios

The following use case descriptions expand on the system's use case diagram. Each description identifies the actor, preconditions, normal flow, alternative flows, and postconditions to provide complete behavioural specifications.

UC-01 — Ingest CSV Data

Use Case ID	UC-01
Name	Ingest CSV Data
Actor(s)	CSV Job Boards (External System), Airflow Scheduler (System Actor)
Trigger	The Airflow daily schedule fires at the configured time, or a Data Engineer manually triggers the DAG.
Preconditions	CSV files are available in the designated landing zone. The Pydantic schema definition is deployed and current.
Normal Flow	1. Airflow triggers the <code>extract_csv</code> task. 2. The system reads the CSV file in configurable chunks using <code>pandas</code> . 3. Each chunk is validated against the Pydantic schema (field presence, data types, format rules). 4. Valid rows are batched and written as raw Parquet to the Bronze ADLS layer. 5. The task completes and reports record counts to the Airflow log.
Alternative Flow	A-1 (Partial rejection): Some rows fail validation. The system routes failed rows to the dead-letter store with their rejection reason and continues processing the remaining valid rows. The task succeeds with a warning.
Exception Flow	E-1 (File not found): The CSV file is absent or inaccessible. The task fails, Airflow marks it as failed, and a notification is sent to the Data Engineer.
Postconditions	Valid job records are persisted as Parquet in Bronze ADLS. Rejected records are stored in the dead-letter table with rejection reasons. Record counts are written to Airflow task logs.
Related Reqs	FR-01, FR-03, FR-04, FR-05, FR-06

UC-02 — Ingest API Data

Use Case ID	UC-02
Name	Ingest API Data
Actor(s)	REST APIs (External System), Airflow Scheduler (System Actor)
Trigger	Airflow triggers the <code>extract_api</code> task as part of the daily DAG run.
Preconditions	REST API credentials are available in the secrets manager. The network is accessible to the external API endpoints.
Normal Flow	1. Airflow triggers the <code>extract_api</code> task. 2. The system sends authenticated HTTP requests to each configured API endpoint. 3. API responses are streamed and parsed record by record. 4. Each record is validated against the Pydantic schema. 5. Valid records are written as raw Parquet to the Bronze ADLS layer.

Alternative Flow	A-1 (Transient HTTP failure): A request fails with a 5xx or timeout. The system applies exponential-backoff retry logic for up to three attempts before marking the record as failed and routing it to the dead-letter store.
Exception Flow	E-1 (API authentication failure): Credentials are invalid or expired. The task fails immediately, logs the error, and Airflow notifies the Data Engineer.
Postconditions	All successfully retrieved API records are persisted as Parquet in Bronze ADLS. Unrecoverable records are stored in the dead-letter table.
Related Reqs	FR-02, FR-03, FR-04, FR-05

UC-03 — Run Silver Transformation

Use Case ID	UC-03
Name	Run Silver Transformation
Actor(s)	Airflow Scheduler (System Actor)
Trigger	The Bronze landing tasks (UC-01, UC-02) complete successfully for the current DAG run.
Preconditions	Bronze Parquet files for the current run date are available in ADLS. DuckDB is configured to access the ADLS Bronze container. dbt Silver models are deployed.
Normal Flow	1. Airflow triggers the dbt_run_silver task. 2. DuckDB reads Bronze Parquet files directly from ADLS without local staging. 3. dbt Silver models execute: cleaning null values, standardising field formats, and deduplicating records by job identifier. 4. Cleaned, staged Silver Parquet files are written to the Silver ADLS layer. 5. dbt logs row counts and test results; the task completes successfully.
Alternative Flow	A-1 (dbt test failure): A dbt data quality test fails. The task is marked as failed. Downstream Gold and PostgreSQL tasks do not execute, preventing corrupted data from reaching the serving layer.
Postconditions	Silver Parquet files containing clean, deduplicated, source-staged records are available in the Silver ADLS layer.
Related Reqs	FR-07, FR-08, FR-10, NFR-03

UC-04 — Run Gold Aggregation

Use Case ID	UC-04
Name	Run Gold Aggregation
Actor(s)	Airflow Scheduler (System Actor)
Trigger	The Silver transformation task (UC-03) completes successfully.
Preconditions	Silver Parquet files for the current run date are available in the Silver ADLS layer. dbt Gold models are deployed.
Normal Flow	1. Airflow triggers the dbt_run_gold task. 2. DuckDB reads Silver Parquet files. 3. dbt Gold models UNION records from all Silver sources and compute five aggregated tables: job title trends, location distributions,

	salary band summaries, skills demand rankings, and daily posting volumes. 4. Gold Parquet files are written to the Gold ADLS layer.
Postconditions	Five aggregated Gold tables are materialised in the Gold ADLS layer, ready for loading into PostgreSQL.
Related Reqs	FR-09, FR-10

UC-05 — Load PostgreSQL Serving Layer

Use Case ID	UC-05
Name	Load PostgreSQL Serving Layer
Actor(s)	Airflow Scheduler (System Actor)
Trigger	The Gold aggregation task (UC-04) completes successfully.
Preconditions	Gold ADLS files are current. PostgreSQL connection credentials are available in the secrets manager.
Normal Flow	1. Airflow triggers the load_postgres task. 2. For each of the five Gold tables, the system truncates the existing PostgreSQL table. 3. The system loads the corresponding Gold Parquet data into the now-empty table. 4. Row counts are verified. The task completes and reports load metrics to Airflow logs.
Alternative Flow	A-1 (Row count mismatch): Loaded row count does not match the Gold source. The system logs an error and marks the task as failed without committing the partial load.
Postconditions	All five Gold tables in PostgreSQL reflect the latest aggregated data. The BI dashboard is immediately up to date.
Related Reqs	FR-11, NFR-05

UC-06 — View Job Trend Dashboard

Use Case ID	UC-06
Name	View Job Trend Dashboard
Actor(s)	Data Analyst
Trigger	The Data Analyst opens the BI dashboard in a web browser.
Preconditions	PostgreSQL Gold tables have been loaded successfully by the most recent pipeline run. The analyst has Read access to the dashboard.
Normal Flow	1. The analyst opens the dashboard. 2. The dashboard queries PostgreSQL and renders default trend visualisations: posting volume time series, top job titles, and top locations. 3. The analyst applies filters: job title, location, date range, salary band. 4. The dashboard re-queries and updates all charts in real time. 5. The analyst views time-series charts, geographic heat maps, and skills frequency rankings.
Alternative Flow	A-1 (No data for filter): The selected filter combination returns no records. The dashboard displays an empty-state message prompting the analyst to widen the filter criteria.

Postconditions	The analyst has reviewed job market trends and can export or share the filtered view.
Related Reqs	FR-12, FR-13

UC-07 — Monitor Pipeline Runs

Use Case ID	UC-07
Name	Monitor Pipeline Runs
Actor(s)	Data Engineer
Trigger	The Data Engineer opens the Airflow UI to check the status of a completed or in-progress pipeline run.
Preconditions	The Airflow web server is running and the engineer has operator-level access.
Normal Flow	1. The engineer navigates to the Airflow DAG view. 2. The system displays the run history with status (success, running, failed) for each run. 3. The engineer clicks a specific run to view task-level status. 4. The engineer opens a task log to inspect record counts, rejected-row counts, and transformation durations. 5. The engineer confirms the pipeline completed successfully.
Alternative Flow	A-1 (Failed task): The engineer identifies a failed task, reads the error message in the task log, and decides to trigger a manual backfill (UC-08) or inspect the dead-letter store (UC-09).
Postconditions	The engineer has full visibility into the run outcome and can take corrective action if required.
Related Reqs	FR-16, NFR-10

UC-08 — Trigger Manual Backfill

Use Case ID	UC-08
Name	Trigger Manual Backfill
Actor(s)	Data Engineer
Trigger	The Data Engineer identifies that one or more historical pipeline runs failed or produced incorrect output.
Preconditions	The engineer has operator-level Airflow access. Source data for the target date range is available or has been corrected.
Normal Flow	1. The engineer navigates to the Airflow DAG page and selects the Backfill option. 2. The engineer specifies the start and end dates for the backfill. 3. Airflow schedules and executes the full pipeline for each missing date in order. 4. The pipeline produces identical Gold and PostgreSQL output as a regular run. 5. The engineer confirms successful completion via the monitoring view.
Postconditions	Historical Gold and PostgreSQL data for the specified date range is corrected and consistent with current data.
Related Reqs	FR-15, NFR-05

UC-09 — Inspect Dead-Letter Records

Use Case ID	UC-09
Name	Inspect Dead-Letter Records
Actor(s)	Data Engineer
Trigger	The pipeline run completes with a non-zero rejected-row count, or the engineer proactively reviews the dead-letter store.
Preconditions	The dead-letter table is populated with one or more rejected records. The engineer has read access to the dead-letter store.
Normal Flow	1. The engineer queries the dead-letter store (via Airflow log or direct database query) to list failed records. 2. The system returns each record with its raw_data and error_message. 3. The engineer identifies the root cause: schema mismatch, missing mandatory field, or upstream source error. 4. If the issue is fixable, the engineer corrects the source data and triggers a manual backfill (UC-08). 5. If the records are non-recoverable, the engineer marks them as resolved and updates the is_resolved flag.
Alternative Flow	A-1 (Fixable batch): The engineer determines the rejection was caused by a temporary upstream schema change. After the source schema is corrected, a backfill re-processes the affected date range.
Postconditions	All dead-letter records have a documented resolution. The is_resolved flag is updated. Recovered records are re-ingested where applicable.
Related Reqs	FR-04, FR-16, NFR-04

4. Use Case Diagram

The use case diagram below illustrates the boundary of the Job Trends Pipeline system and all interactions between external actors (CSV Job Boards, REST APIs, Airflow Scheduler, Data Analyst, and Data Engineer) and the system's use cases. It serves as a high-level map of who does what with the system.

Diagram Note

The use case diagram maps nine system use cases across five actors. Solid lines indicate direct associations; dashed «include» arrows show mandatory dependencies between use cases (e.g., Ingest CSV always includes Validate & Reject Records); the «extend» relationship on Inspect Dead-Letter Records indicates it is triggered conditionally when validation failures occur.

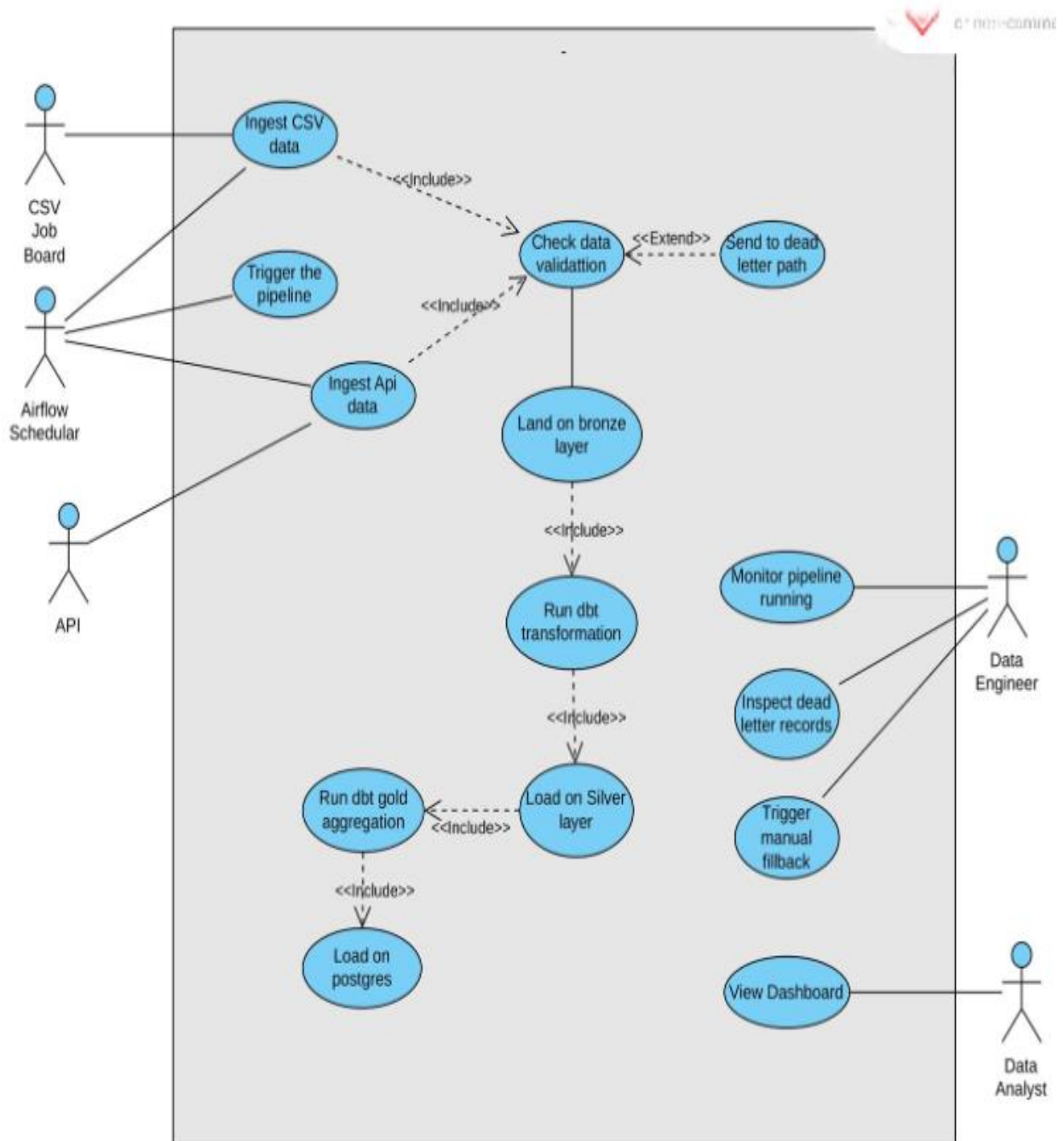
Actors in the diagram:

- CSV Job Boards — External source system supplying raw CSV job listings.
- REST APIs — External source providing live job data via HTTP endpoints.
- Airflow Scheduler — Internal system actor that triggers all automated pipeline tasks daily.
- Data Analyst — Human actor who queries and interacts with the BI dashboard.
- Data Engineer — Human actor responsible for monitoring, backfilling, and inspecting failures.

Use cases in the system boundary:

- Ingest CSV Data (UC-01) — Includes: Validate & Reject Records, Land to Bronze (Parquet)

- Ingest API Data (UC-02) — Includes: Validate & Reject Records, Land to Bronze (Parquet)
- Run Silver Transformation (UC-03)
- Run Gold Aggregation (UC-04)
- Load PostgreSQL Serving Layer (UC-05)
- View Job Trend Dashboard (UC-06)
- Monitor Pipeline Runs (UC-07)
- Trigger Manual Backfill (UC-08)
- Inspect Dead-Letter Records (UC-09) — Extends: Validate & Reject Records



shows the system-level use case diagram with all actors and their interactions across

5. Data Flow Diagram (DFD)

The Data Flow Diagram below shows how data moves through the pipeline from its two external sources to the serving layer. It illustrates processes, data stores, and the data flows that connect them, giving a clear picture of the system’s data transformation path from raw ingestion through Bronze, Silver, and Gold layers to the final PostgreSQL and BI serving layer.

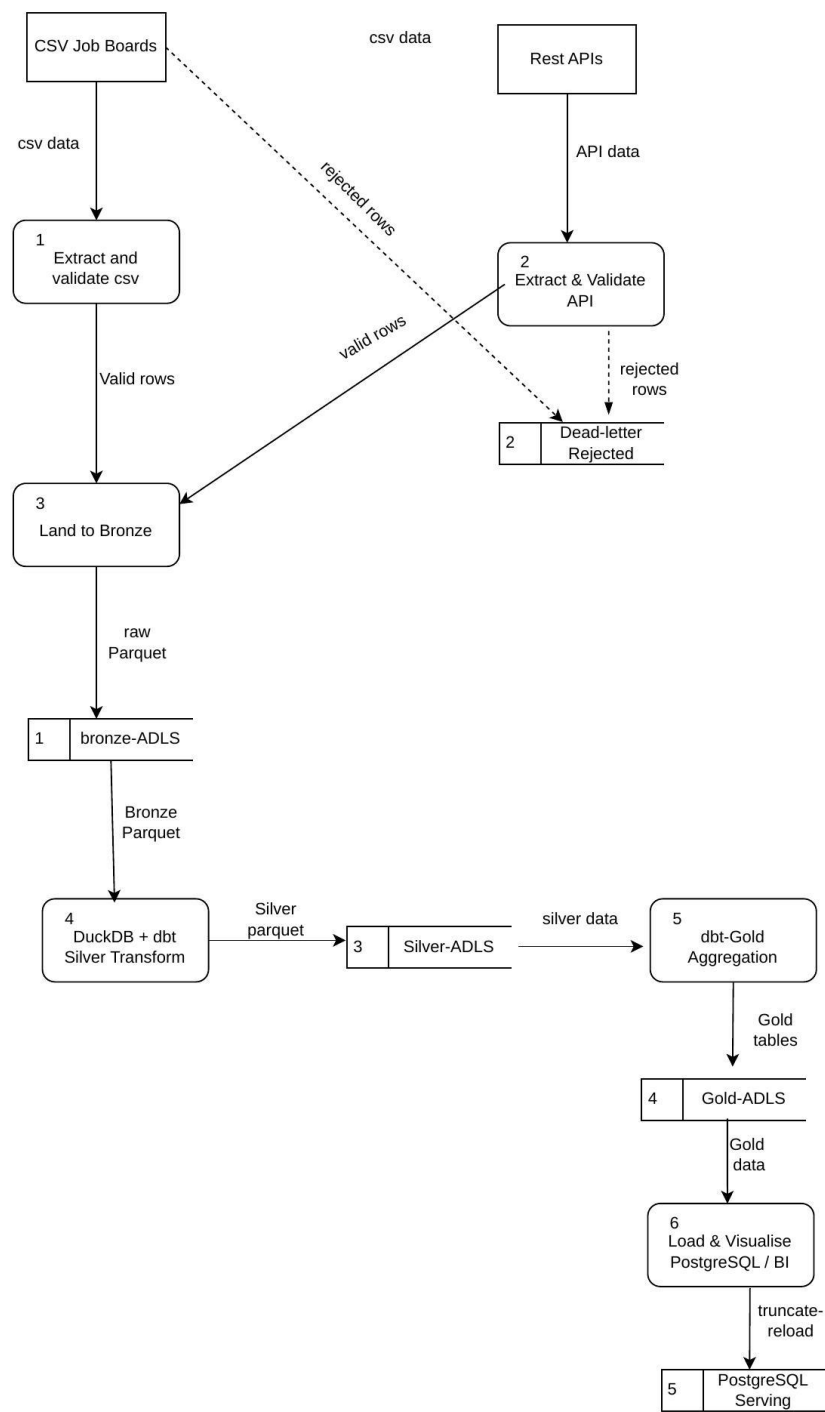


Figure 5.1 — Data Flow Diagram: Job Trends Pipeline (Level 1)

Key flows shown in the DFD:

- CSV Job Boards → Process 1 (Extract & Validate CSV) → Bronze ADLS: Raw job records from CSV files are validated and landed as Parquet.
- REST APIs → Process 2 (Extract & Validate API) → Bronze ADLS: API job records undergo the same validation path; rejected rows are routed to the Dead-Letter store (Process 2 / Store 2).
- Bronze ADLS → Process 4 (DuckDB + dbt Silver Transform) → Silver ADLS: Bronze Parquet is cleaned and deduplicated.
- Silver ADLS → Process 5 (dbt Gold Aggregation) → Gold ADLS: Silver records are aggregated into five Gold tables.
- Gold ADLS → Process 6 (Load & Visualise) → PostgreSQL Serving: Gold data is truncate-reloaded into PostgreSQL, which powers the BI dashboard.

6. Activity Diagrams

Activity diagrams model the step-by-step workflow of key system processes, showing decision points, parallel activities, and alternative paths. Two activity diagrams are included: the main pipeline execution flow and the dead-letter monitoring and reprocessing flow.

6.1 Main Pipeline Execution Flow

This activity diagram shows the complete end-to-end pipeline triggered by Airflow. It captures parallel ingestion from CSV and API sources, the validation decision gate that splits valid rows from rejected rows, the sequential Bronze-Silver-Gold transformation chain, and the final load to PostgreSQL and rd.

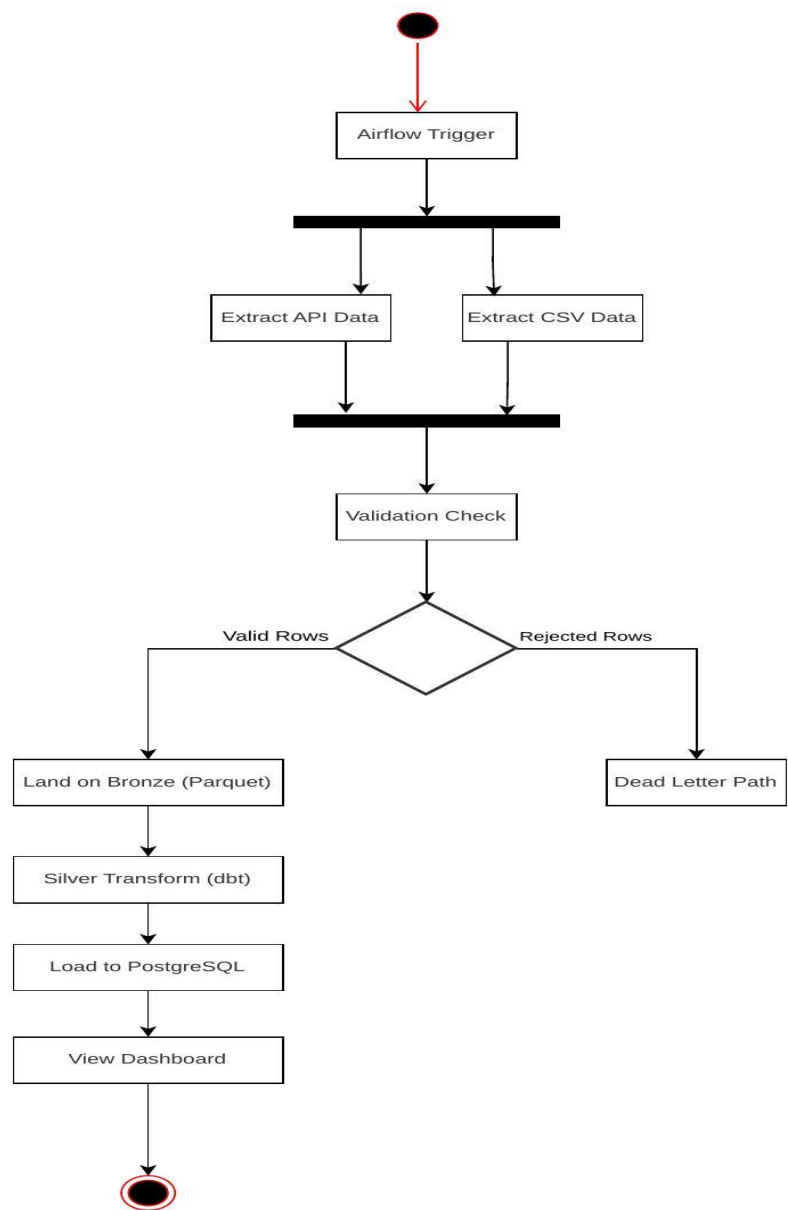


Figure 6.1 — Activity Diagram: Main Pipeline Execution (Airflow Trigger to Dashboard)

6.2 Dead-Letter Monitoring and Manual Reprocessing Flow

This activity diagram shows what happens after a pipeline run produces rejected records. The Data Engineer monitors the dead-letter queue, decides whether each failure is fixable, and follows one of two paths: triggering a manual backfill with corrected source data, or discarding and logging irrecoverable records

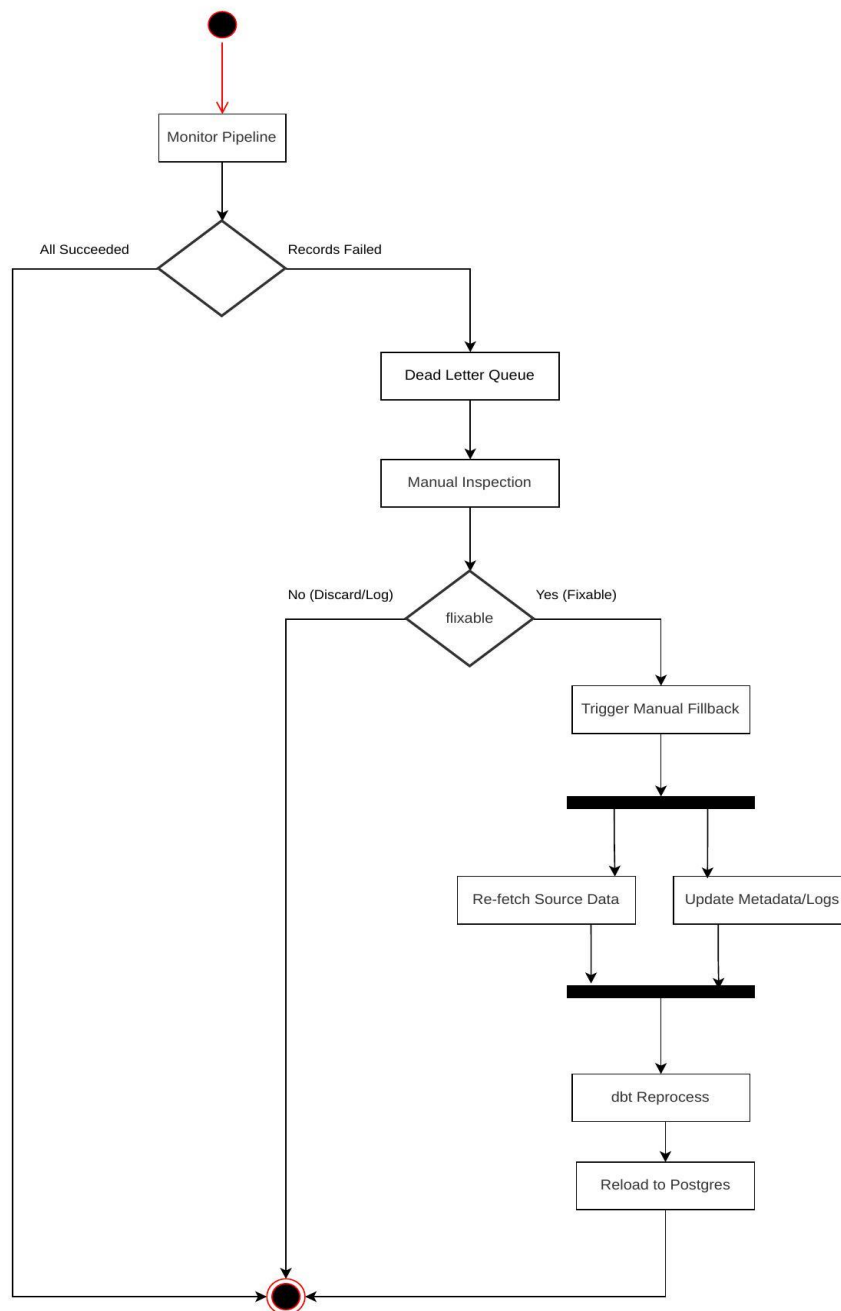


Figure 6.2 — Activity Diagram: Dead-Letter Monitoring and Manual Backfill

7. Sequence Diagrams

Sequence diagrams show the time-ordered interactions between system components for two key scenarios. They complement the activity diagrams by specifying which component sends which message to which other component, in what order.

7.1 Pipeline Trigger and Bronze Landing Sequence

This sequence diagram covers the first phase of the pipeline: Airflow triggers parallel ingestion from the CSV source and the API source; both sources send raw records to the Validator; the Validator passes valid records to the Bronze ADLS layer; and Airflow confirms the Bronze landing is complete before signalling that the pipeline can proceed to transform

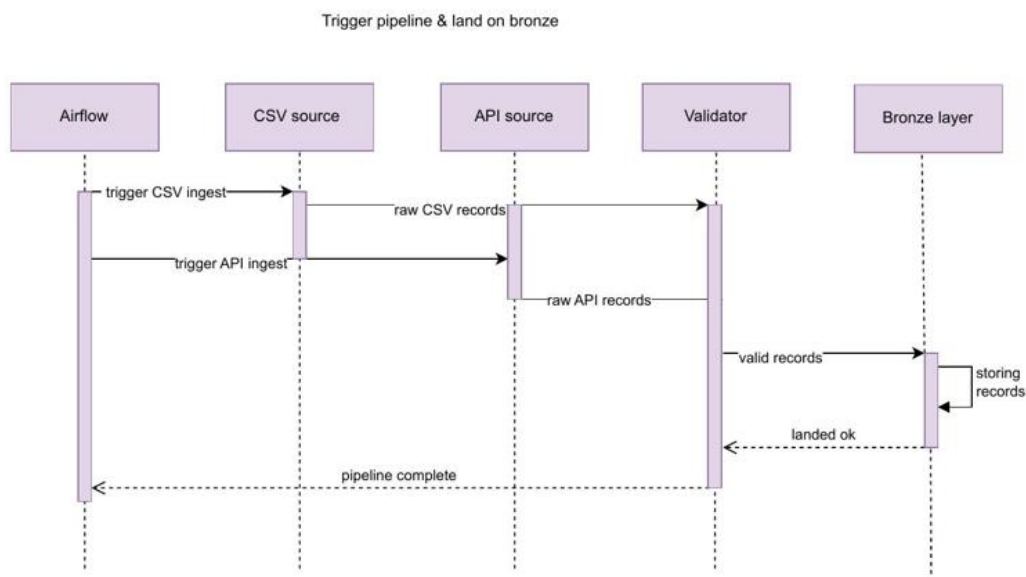
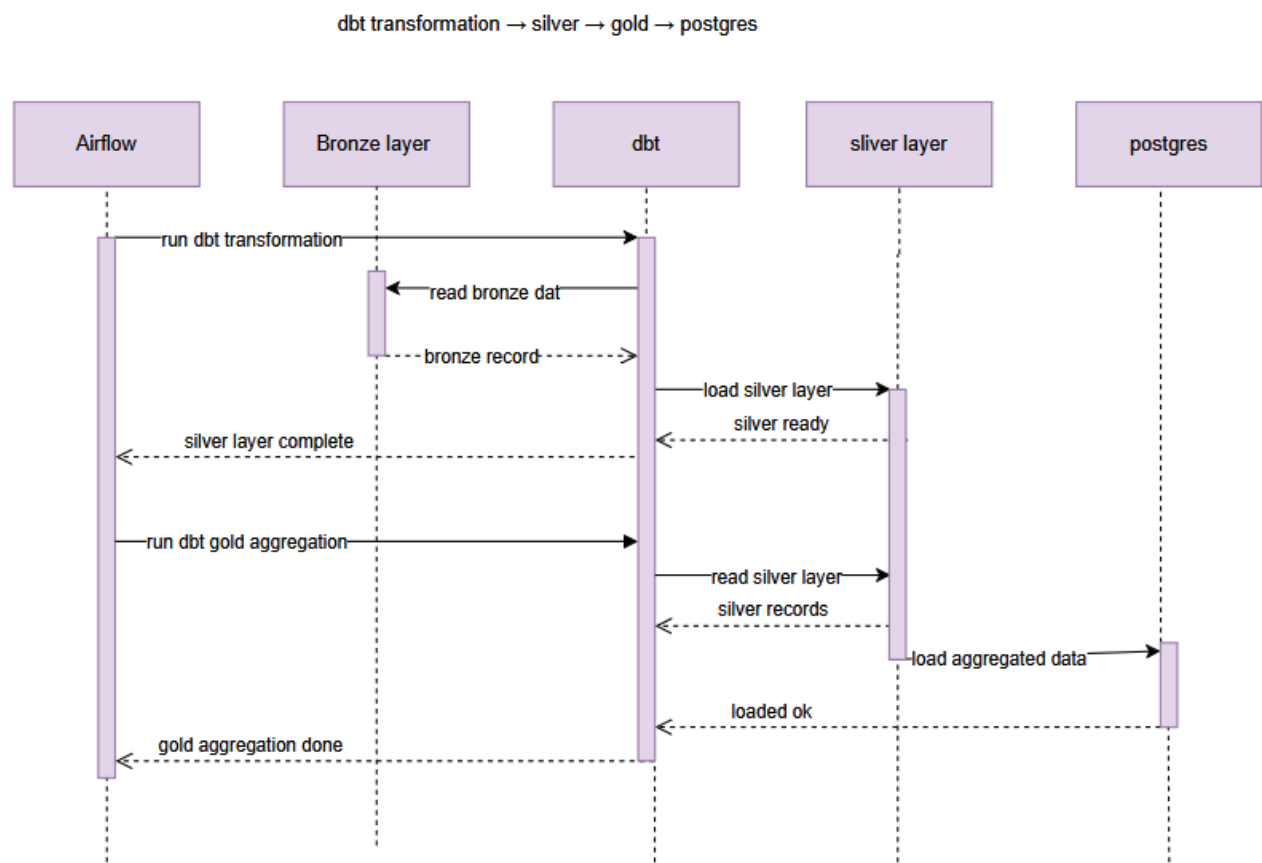


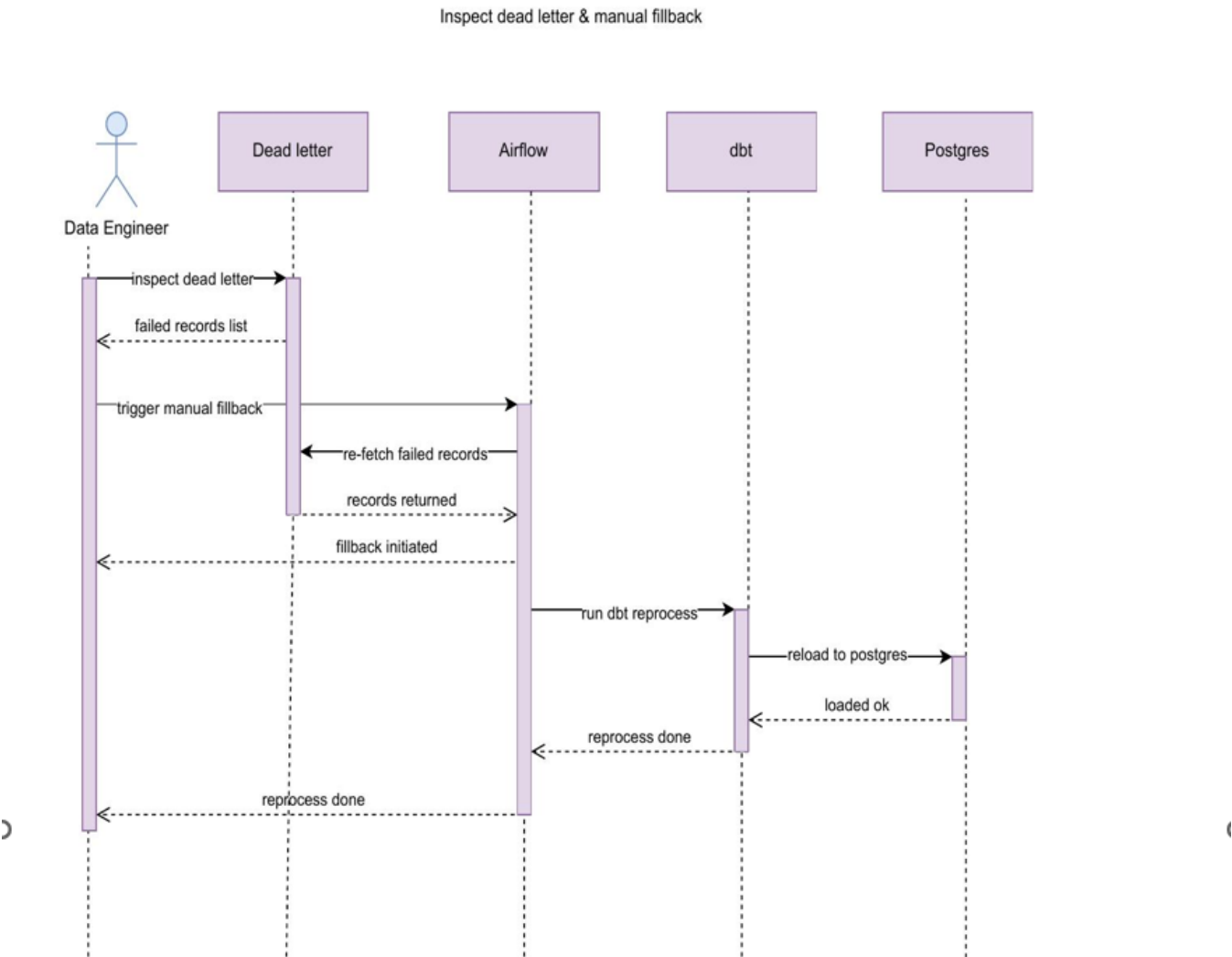
Figure 7.1 — Sequence Diagram: Pipeline Trigger and Bronze Landing

7.2 DBT transformation and ,Silver landing, Aggregation , Gold landing and load to postgres



7.3 Inspect Dead-Letter and Manual Backfill Sequence

This sequence diagram shows the interaction flow when a Data Engineer responds to failed records. The engineer queries the dead-letter store to retrieve failed records, triggers a manual backfill via Airflow, which in turn re-fetches the source records, runs dbt reprocessing through Silver and Gold, and reloads the corrected data into PostgreSQL.



8. Entity Relationship Diagram (ERD)

The ERD below defines the logical data model for the Gold serving layer and the supporting operational tables. It shows all entities, their attributes with data types, primary keys, foreign keys, and the cardinality relationships between them. The model follows a star-schema pattern centred on the `FACT_JOB_POSTING` table, with dimension tables for company, location, search city, and data source.

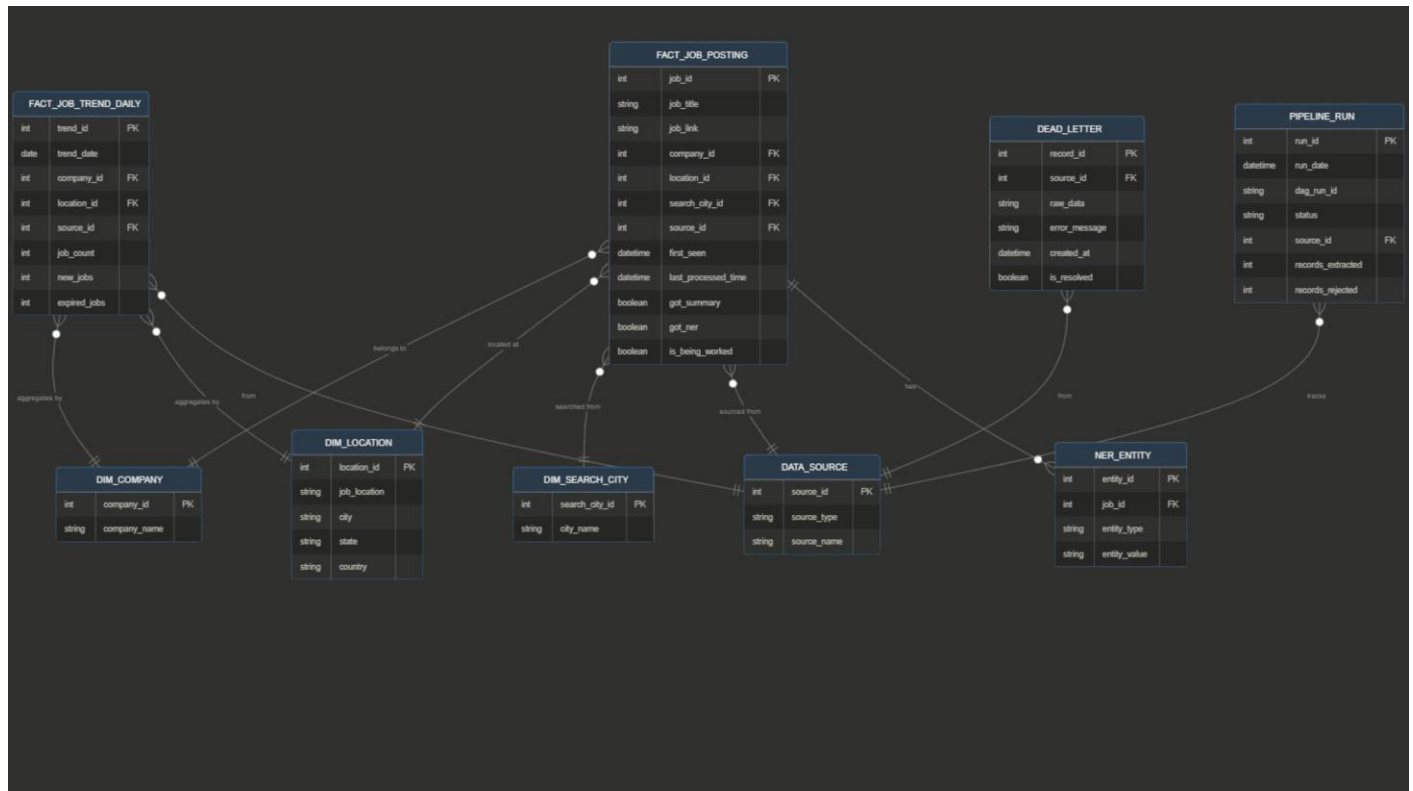


Figure 8.1 — Entity Relationship Diagram: Job Trends Data Model

Entity summary:

- **FACT_JOB_POSTING** — Central fact table. One row per unique job listing. Foreign keys link to `DIM_COMPANY`, `DIM_LOCATION`, `DIM_SEARCH_CITY`, and `DATA_SOURCE`. Boolean flags track NLP enrichment status (`got_summary`, `got_ner`).
- **FACT_JOB_TREND_DAILY** — Daily aggregated trend fact. Stores `job_count`, `new_jobs`, and `expired_jobs` per company, location, and source combination.
- **DIM_COMPANY** — Dimension table with `company_id` and `company_name`.
- **DIM_LOCATION** — Dimension table with `job_location`, `city`, `state`, and `country`.
- **DIM_SEARCH_CITY** — Dimension table recording the city used as the search parameter when the job was fetched.
- **DATA_SOURCE** — Tracks whether a record originated from a CSV file or an API endpoint (`source_type`, `source_name`).
- **DEAD_LETTER** — Operational table storing rejected records with `raw_data`, `error_message`, and `is_resolved` flag.
- **PIPELINE_RUN** — Audit table logging each Airflow DAG run with `status`, `records_extracted`, and `records_rejected` counts.

- **NER_ENTITY** — Stores named entities extracted from job descriptions via NLP enrichment, linked to **FACT_JOB_POSTING** by **job_id**.